Prof. Dr. Erwin Mayer

Fakultät für Elektrotechnik
und Informationstechnik

Praktikum Verteilte Systeme

Versuch 2

RPC Programmierung



Was Sie in diesem Versuch lernen:

- Vertrautwerden mit einer RPC-basierten-Entwicklungs- und Laufzeitumgebung
- Einsatz einer IDL als Beschreibungssprache für verteilte Schnittstellen
- Kennenlernen eines RPC Stub- und Skeleton-Compilers
- Entwickeln eines RPC Cients und eines RPC-Servers
- Absprache von Schnittstellen im Team und verteilte Komponentenentwicklung
- Testen der RPC-Anwendung

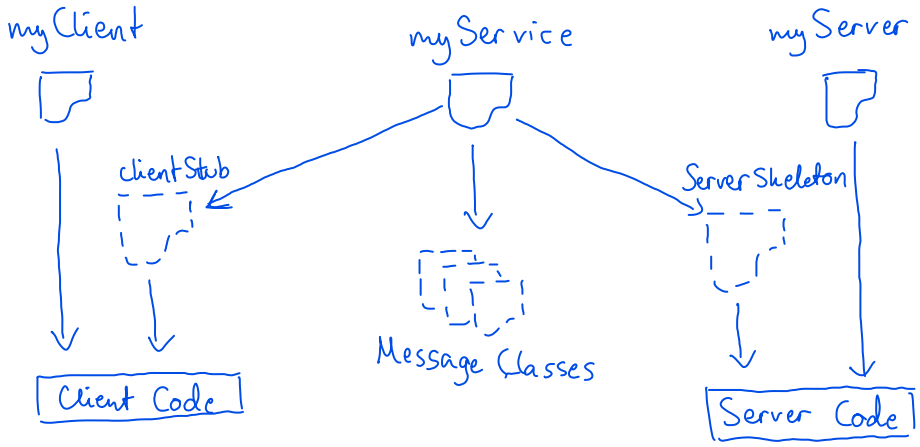
Voraussetzug zur Teilnahme: Pre-Lab Aufgaben komplett durchgeführt (mit Nachweis) !

23/04/26

1 PRE-LAB-Fragen

Name	Matrikelnummer	Semester

1. Skizzieren Sie, beginnend mit einem gRPC-proto-File, welche Teile der gRPC-Client-Server-Architektur aus dem proto generiert werden (gestrichelt) und welche selbst entwickelt werden müssen (nicht gestrichelt). Hinweis: Folie aus Vorlesung.



2. Definieren Sie über ProtoBuf v3 eine Schnittstelle für ein Serverobjekt, welches eine Mitgliederliste verwaltet (Listenelemente: Mitgl.-Nr., Name, Telefonnnummer. Operationen: Mitglied dazunehmen, Mitglied entfernen, Gesamte Liste abfragen) . (Hinweis: Wenn Sie keinen Parameter oder Returnwerte haben, müssen Sie dafür eine Empty-Message anlegen; es gibt hierfür google.protobuf.Empty)

```
message Member{
  int32 member_nr = 1;
  string name = 2;
  string phonenumber = 3;
}

message MemberList{
  repeated Member members = 1;
}
```

3. Schauen Sie sich im Internet das gRPC-Proto-File https://github.com/grpc/grpc/blob/v1.35.0/examples/protos/route_guide.proto an. Machen Sie sich schlau, was die Anwendung macht. Weitere Erläuterungen zur Anwendung finden Sie unter https://grpc.io/docs/languages/java/basics/. Vervollständigen Sie unten auf dieser Seite den rpc-Typ, beschreiben Sie knapp, wofür jede der 4 Methoden dient und wofür das Streaming jeweils gebraucht wird.

1. rpc GetFeature (Point) returns (Feature) {}

   Was macht es?

   holt ein feature an der spezifizierten position

2. rpc ListFeatures (Rectangel) returns (stream Feature) {}

   Was macht es?

   holt alle features in dem rectangle

   Wofür genau wird Streaming gebraucht?

   bei mehrfachen / vielen daten

3. rpc RecordRoute(stream Point) returns (RouteSummary) {}

   Was macht es?

   nimmt ein stream an punkten an auf einer route die durchquert wird. gibt eine routeSummary nach durchquerung zurück

   Wofür genau wird Streaming gebraucht?

   alle punkte einsammeln und ganz am ende was zurückgeben

4. rpc RouteChat (stream RouteNote) returns (stream RouteNote) {}

   Was macht es?

   akzeptiert stream von routeNotes während durchquerung und nimmt auch noch andere RouteNotes auf (von anderen usern)

   Wofür genau wird Streaming gebraucht?

   asynchrone kommunikation, client muss nicht auf antwort warten und server kann allzeit an alle clients pushen (solange sie erreichbar sind)

2 Praktikum

2.1 Einführung

In diesem Versuch sollen Sie lernen, ein RPC-Framework zur Implementierung einer verteilten Anwendung einzusetzen. Allerdings soll die Anwendung dieses Mal nicht peer-to-peer-basiert sein, sondern über einen zentralen RPC-basierten Server implementiert werden.

Für den Laborversuch werden sie einsetzen:

- Linux als Betriebssystem
- Java als Programmiersprache
- gRPC (Google RPC) als Middleware
- IntelliJ als IDE

2.2 Gruppenbildung

Wie in Laborversuch 1 arbeiten Sie in Gruppen zu je 3 Studierenden. JEDER der 3 Teilnehmer muss dabei sowohl die Client- als auch die Server-Komponente entwickeln.

1. Die Gruppe definiert gemeinsam die Schnittstellen zwischen den verteilten Chatprozessen in Form einer gRPC IDL Spezifikation (→ Resultat: 1 proto-File).
2. Die Gruppe diskutiert zentrale Entwurfsentscheidungen

   CHECKPOINT → proto-File dem Laborleiter zeigen
3. Die Gruppe bringt die Web-Beispielprogramme gemeinsam zusammen zum Laufen (d.h. baut zusammen die Infrastruktur auf)

Gruppenarbeit

4. Jeder implementiert seine Komponente (= BEIDES Client und Server).
   Kein Side-by-Side-Programming !

Einzelarbeit

5. Die Gruppe führt das Testen des Verteilten Systems gemeinsam durch

2.3 Aufgabenstellung

Realisieren Sie eine verteilte CHAT-Anwendung mit gRPC.

Die Chat-Anwendung soll als Beispiel einer Service-orientierten Architektur (SOA) dienen.

- Teilaufgabe 1: Klassische Client-Server-Architektur
- Teilaufgabe 2: Erweiterung um Service-Chaining (History Server)

Labortermin 1 realisiert die erste Teilaufgabe, Labortermin 2 den zweiten Teil.

2.4 Lastenheft – Teil 1

Funktionale Anforderungen

1. Die Anwendung soll N (2 < N <= 10) Chat-Teilnehmer unterstützen
2. Jeder Teilnehmer wird über ein Konsolkommando gestartet
3. Jedes Teilnehmerprogramm fragt zunächst den Chat-Teilnehmer nach einem Nickname.
4. Nach der entsprechenden Eingabe wird der User beim Server registriert und alle bereits im CHAT befindlichen Teilnehmer erhalten eine Nachricht "xxxx ist dem CHAT beigetreten".
5. Falls der Nickname schon existiert erhält der Teilnehmer eine Fehlermeldung.
6. Ein Teilnehmer hat jederzeit die Möglichkeit die Teilnehmerliste abzufragen.
7. Nach Registrierung soll der Teilnehmer in einer Endlosschleife Chat-Messages eingeben können.
8. Jede durch Return abgeschlossene Eingabe wird mit dem Nicknamen des Senders als Präfix bei allen anderen bereits im Chat befindlichen Teilnehmern angezeigt (also z.B. „FRITZ: Hallo zusammen !“..
9. Mit dem Kommando „!Exit“ verlässt ein Teilnehmer den Chat.
10. Das Verlassen eines Teilnehmers wird ebenfalls allen anderen Teilnehmern mitgeteilt.

Nichtfunktionale Anforderungen

11. Ein einzelner CHAT-Server soll den Verteilvorgang der Nachrichten realisieren.
12. Wo möglich und sinnvoll sollen gRPC-Streaming-Mechanismen verwendet werden (d.h. „stream“-Parameter im proto-File).
13. Wo möglich soll das „oneof“-Syntaxelement von protobuf eingesetzt werden.

2.5 Vorgehensweise

Gruppenarbeit:

Sie sollen das Quickstart-Beispiel aus dem Internet
(https://grpc.io/docs/languages/java/quickstart/) zum Laufen bringen.

Vorgehensweise:

1. In IntelliJ:

   `New project` → `Gradle` auswählen → `Kotlin` auswählen
   // liefert standard Java Projekt mit initialem build.gradle.kts

2. `build.gradle.kts` ersetzen mit Inhalt des `build.gradle.kts` aus Moodle

3. Neues Verzeichnis `proto` unter `src/main` anlegen (siehe Anhang A)

4. Neue Datei `hello.proto` in diesem Verzeichnis anlegen mit Inhalt aus Internet

   `hello.proto` steht unter https://github.com/grpc/grpc-java/tree/master/examples/src/main/proto

5. Mit `./gradlew clean build` Projekt erstmalig bauen
   (Hinweis: Client und Server fehlen noch, aber die Stubs und Skeletons und die Verzeichnisstruktur werden jetzt schon mal aufgebaut)

6. In das Verzeichnis `src/main/java/org.example` die Anwendung kopieren, d.h. `ChatClient.java` und `ChatServer.java`.

   `ChatClient.java` und `ChatServer.java` stehen unter:
   https://github.com/grpc/grpc-java/tree/master/examples/src/main/java/io/grpc/examples/helloworld .

7. Mit `./gradlew clean build` Projekt nun vollständig bauen

8. Erst Server, dann Client in IntelliJ starten.

ACHTUNG: Der Quickstart-Code ist nicht direkt für IntelliJ vorkonfiguriert. Bei Problemen nutzen Sie die „Debugging Hints“ in Abschnitt 2.7

Ab hier Einzelarbeit !

Verwenden Sie IntelliJ zum Editieren und Bauen der Anwendung.

1. **Verzeichnisse in denen Sie etwas ändern müssen**.

   - `project-root/src/main/proto/` -- für die eigenen PROTO-Files

   - `project-root/src/main/java/…../myClientMain..`
     -- für die zu erstellenden Anwendungen

     Hinweis: Sie müssen ein eigenes Verzeichnis mit passendem Namen anlegen, in dem Sie ihre Anwendungen ablegen.

2. Verzeichnisse in denen Sie NICHTS ändern müssen (generierter Code), die Sie aber zum **Nachschlagen der Methoden verwenden können, die Sie in ihrer Anwendung brauchen !)**

   - `project-root/build/generated/source/proto/main/grpc/…/MyService.java`
     -- für die generierten Stubs/Skeletons

   - `project-root/build/generated/source/proto/main/java/…/myMsgType.java`
     -- für die generierten Message-Klassen

2.6 Hilfestellungen und Tipps aus dem Web

Zum Umgang mit der **Protobuf-IDL** eignen sich die folgenden Dokumente:

https://developers.google.com/protocol-buffers/docs/proto3
https://developers.google.com/protocol-buffers/docs/javatutorial

Ein **weiterführendes (Java-)Tutorial**, welche unter anderem auch bereits Streaming Elemente beinhaltet finden Sie unter:

https://grpc.io/docs/languages/java/basics/ | muss ebenfalls leicht angepasst werden !

Den kompletten Source Code einschließlich Beispiel-Client und Beispiel-Server für das obige Basics-Tutorial finden Sie unter:

https://github.com/grpc/grpc-java/tree/master/examples/src/main/java/io/grpc/examples/routeguide

Die komplette **Java API Reference** finden Sie unter:

https://grpc.github.io/grpc-java/javadoc/

2.7 Sonstige Tipps und Hilfestellungen

Wenn Sie „aufräumen“ oder „rücksetzen“ wollen:

- `./gradlew clean` löscht alle generierten Files und räumt auf.
- `./gradlew clean build` baut das Projekt von Grund auf.
- `./gradlew clean build –refresh-dependencies` evaluiert zusätzlich die Abhängigkeiten.
- `File→Invalidate Caches & Restart` setzt IntelliJ zurück

Wenn Sie unsicher sind, wie Sie in der Anwendung Nachrichten bauen:

- Schauen Sie sich den generierten Code für ihre Messages unter `build/generated/source/proto/main/java` an.
- Tipp: Aus einem Feld `my_first_field` werden Zugriffsmethoden der Art
  - `getMyFirstField`
  - `setMyFirstField`
  - ….

Wenn Sie unsicher sind, welche Middleware-Methoden Sie in der Anwendung zur Verfügung haben:

- Schauen Sie sich die generierten Stubs und Skeletons unter `build/generated/source/proto/main/gprc` an.
- Tipp: Aus einem RPC `my_first_rpc` wird eine Methode `myFirstRpc`
  Tipp: Aus einem RPC `MyRPC` wird eine Methode `myRPC`
- Calls für den asynchronen (nonblocking) Stub, welche Sie auf Clientseite aufrufen müssen finden Sie unterhalb des Inline-Kommentars

  ```
  /* A stub to allow clients to do asynchronous rpc calls to service … */
  ```

- Falls Sie auf Clientseite einen synchronen (blocking) Stub verwenden, finden Sie den passenden Call unterhalb des Inline-Kommentars

  ```
  /* A stub to allow clients to do synchronous rpc calls to service ……
  ```

- Die Interface Definition des Server Skeletons, welches Sie auf Serverseite implementieren müssen (die @overrides), sind unterhalb des Inline-Kommentars

  ```
  /* public interface AsyncService { …
  ```

Debugging Hints

1. Wenn Projekt nicht kompiliert oder Build fehlschlägt
   → checken ob falsche Java-Version
   → ggf. Projekt in IntelliJ auf passende SDK-Version einstellen

2. „cannot find symbol“ / Import-Fehler oder Ähnliches
   → checken ob Package-Namen übereinstimmen
   - proto-File (option java_package)
   - Java-Code (Client/Server)
   - build.gradle.kts

3. Generierte Klassen werden nicht gefunden
   →In einigen Fällen erkennt IntelliJ den generierten Code nicht automatisch als Source-Code. Man muss ihn dann manuell als „generated source“ kenntlich machen:

   - In IntelliJ den Projektbaum (links) öffnen
   - Navigiere zu „build/generated/source/proto/main/java”
   - Rechtsklick auf den Ordner java
   - Mark Directory as → Generated Sources Root
   - Dasselbe wiederholen für build/generated/source/proto/main/grpc

4. Beim Testen beachten: Wenn Sie die komplette RPC-Ausführung beobachten wollen, sollten Sie den Client NICHT beenden. Für das Debugging, machen Sie beim initialen Testen z.B. einen SLEEP nach erfolgtem RPC-Aufruf.

5. gRPC-Methoden oder Message-Klassen fehlen
   → Ursache: Codegenerierung wurde nicht ausgeführt
   → Lösung:
   Projekt mit „./gradlew clean build“ neu bauen

6. Client bekommt keine oder unvollständige Antworten
   → Ursache: Stream wird nicht korrekt beendet
   → Lösung:
   onCompleted() aufrufen (bei Streaming unbedingt notwendig)

2.8 Zweiter Labortermin

Der zweite Labortermin strukturiert sich in 3 Teile

Abschnitt 1: System Testing

Sie sollen ihre (funktionsfähige und bereits lokal getestete) grpc-Anwendung mit den anderen Gruppenmitgliedern testen. Jeder der Teilnehmer stellt abwechselnd den Server, während 3 Clients gleichzeitig auf ihn zugreifen.

Denken Sie daran, dass auf Serverseite die Firewall für den Server-Port freigeschaltet werden muss ! (sudo firewall-cmd --add-port=……/tcp)

Falls nötig, korrigieren einzelne Mitglieder danach noch ihre Anwendung.

Abschnitt 2: Analyse

Anschließend sollen Sie übertragene Nachrichten mit ihren Inhalten analysieren (siehe Vorlesungsfolie 7-10). Verwenden Sie hierzu Wireshark (insbesondere das unterste Wireshark Fenster mit Hex-Inhalten). Dekodieren Sie mindestens ein String-Feld (beliebiger Inhalt) und mindestens ein Integer-Feld mit einem Wert > 128 (**zur Not Programm temporär abändern um zu triggern dass diese Inhalte vorhanden sind**).

Hinweis: Die genaue Position einer Feldbeschreibung im Hex-Bereich von Wireshark zu ermitteln ist nicht ganz einfach. Suchen Sie die Beschreibung, indem Sie z.B. vom Nachrichtenende her beginnen und die zu erwartende Länge und den erwarteten Inhalt der Kodierung mit einbeziehen.

Feld 1 (String-Typ)

Feldname (aus proto-File): ChatMsg - text

Binärcode (in hex): … 0a 04 4c 69 6e 68 | 12 04 4d 6f 69 6e

Analyse (wie dekodieren sich die Hexwerte zu einem Inhalt):

Feld 2 (Integer-Typ):

Feldname (aus proto-File): ChatMsg - bla

Binärcode (in hex): … 18 90 03

Analyse:

Abschnitt 3: Weiterentwicklung

Danach soll die Anwendung ein wenig erweitert werden, indem ein sog**. Service-Chaining** zur Realisierung eines History-Servers realisiert wird.

Implementieren Sie folgende zusätzliche Lastenheft-Elemente:

Lastenheft Teil 2:

1.-13. wie bisher

14. Über einen Stream soll der Hauptserver alle Einträge im Chat auf einen „History-Server“ spiegeln.

15. Ein neuer Client soll mit einer History des bisherigen Chats versorgt werden.

16. Der Hauptserver verwendet zu diesem Zweck den History-Server.

Nichtfunktionale Anforderungen

17. Sie können 1 proto-File verwenden aber wo nötig und sinnvoll mehrere Services.

18. Insbesondere ist der History-Server als eigenständiger Service zu realisieren, der vom Chat-Server verwendet wird. Der Chat-Server soll ein Client des History-Servers sein. (als ein Beispiel für „service chaining“).

19. Chat-Clients kommunizieren weiterhin ausschließlich mit dem Chat-Server.

2.9 Abgabe (Post-Lab)

Teil 1

Laden Sie ein ZIP-File mit folgenden Inhalten nach Moodle hoch:

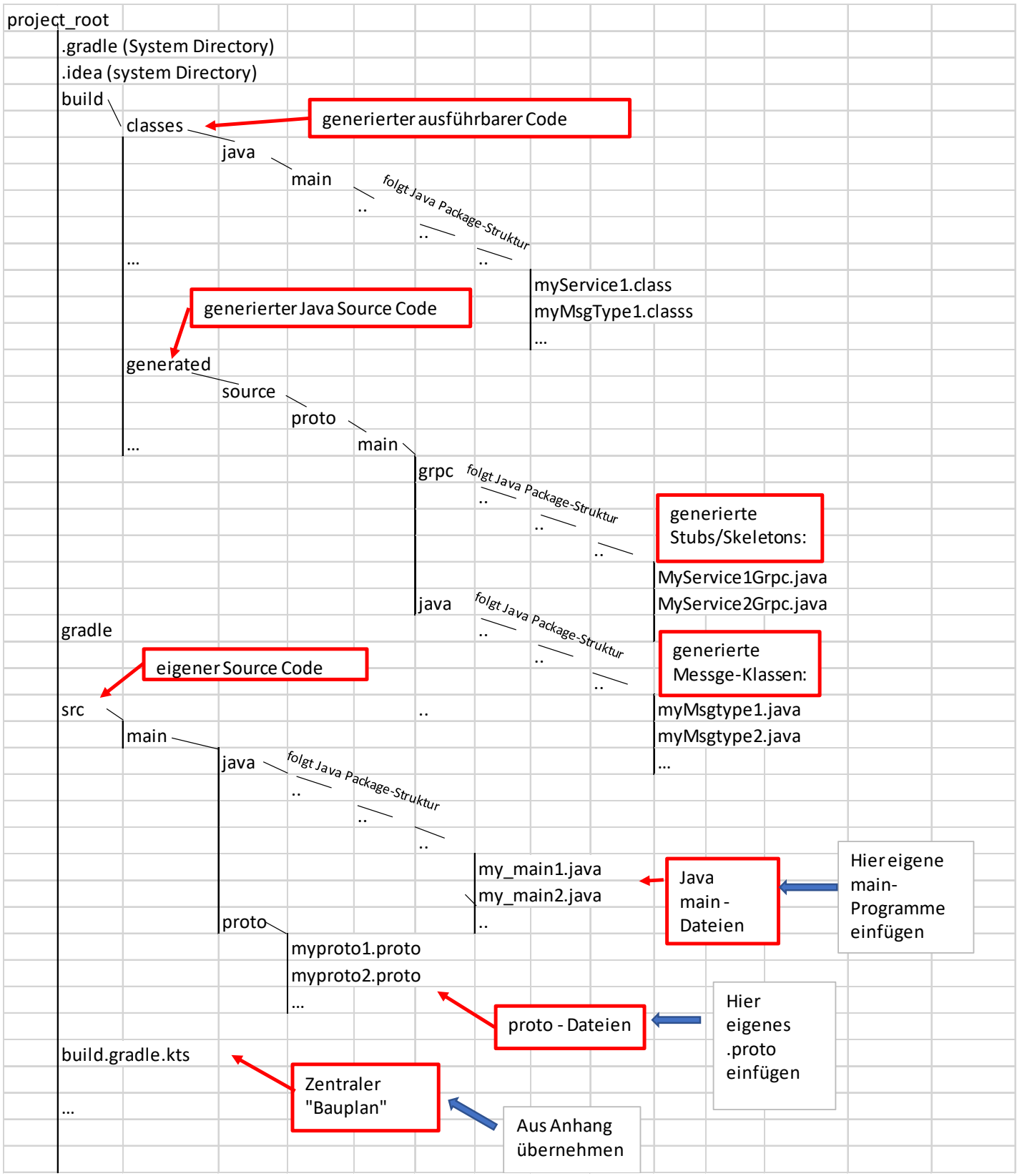
1. IDL-File (.proto)
2. Source Code von Client und von Server
3. Screenshot **"build.jpeg"** von Ihrem Build
   (d.h. von der Ausgabe von "./gradlew clean build")
4. Screenshot **"run.jpeg"** der laufenden Anwendung nach Eingaben von mindestens 3 verschiedenen Clients, die nach und nach hinzukommen, chatten und den Chat sukzessive wieder verlassen.

Teil 2

Bitte laden Sie in ZIP-File mit den folgenden Files nach Moodle hoch:

1. Erweitertes IDL-File (.proto)
2. Endgültiger Source Code von Client, Server und History-Server
3. Analyse-Output für das String- und Integer-Feld.
4. Screenshot **"build.jpeg"** von Ihrem Build
5. Screenshot **"run.jpeg"** der laufenden Anwendung mit 1 Java Server, 3 Java Clients, 1 History-Server jeweils mit mehreren Eingaben (Login, Chat, Exit – mit deutlicher Sichtbarkeit der Verwendung der Historie).

2.10 ANHANG A: gRPC Java Verzeichnisstruktur



2.11 ANHANG B: gradle Build-File

Das folgende gradle build file (gradle.build.kts) steht in Moodle zur Verfügung.

```
// Hinweis: Die angegebenen Versionsnummern sind auf die Laborumgebung
// abgestimmt. Bei Verwendung auf privaten Rechnern („zuhause“) kann es
// erforderlich sein, neuere Versionen der Abhängigkeiten und der Java-
// Version zu verwenden.

plugins {
    id("java")
    id("com.google.protobuf") version "0.9.4"
}

group = "org.example"  //Achtung: Package-Name anpassen !
version = "1.0.0"
java.sourceCompatibility = JavaVersion.VERSION_11  //zuhause ggf. VERSION_17

repositories {
    mavenCentral()
}

dependencies {  //Hinweis: zuhause ggf. neuere Versionen verwenden
    implementation("io.grpc:grpc-netty-shaded:1.71.0")
    implementation("io.grpc:grpc-protobuf:1.71.0")
    implementation("io.grpc:grpc-stub:1.71.0")
    implementation("javax.annotation:javax.annotation-api:1.3.2")

    testImplementation("junit:junit:4.13.2")
}

protobuf {
    protoc {
        artifact = "com.google.protobuf:protoc:3.25.1" //zuhause ggf. 4.32.1
    }
    plugins {
        create("grpc") {
            artifact = "io.grpc:protoc-gen-grpc-java:1.71.0" //ggf. anpassen
        }
    }
    generateProtoTasks {
        all().forEach { task ->
            task.plugins {
                create("grpc") {}
            }
        }
    }
}

tasks.named<ProcessResources>("processResources") {
    exclude("**/*.proto")
    duplicatesStrategy = DuplicatesStrategy.EXCLUDE
}

sourceSets {
    main {
        proto {
            srcDir("src/main/proto")
        }
    }
}
```

2.12 Anhang C: IntelliJ Installation

Linux-Variante von IntelliJ herunterladen (`www.jetbrains.com/de-de/idea/download`)
Auswahl: `.tar.gz` (Linux)

→ liegt jetzt im Download-Verzeichnis (`/home/student/Downloads`)

```
sudo tar -xvzf ~/Downloads/idea-2026*.tar.gz -C /opt
```

```
sudo chown -R student:users /opt/idea-*
```

```
sudo ln -s /opt/idea*/bin/idea /usr/local/bin/idea
```

Danach:

IntelliJ einfach starten mit "`idea`" von beliebiger Linux Shell

2.13 ANHANG D: Proto3 Syntaxbeispiele

```
syntax = "proto3";

// Java-spezifisch
package my.package.name;
option java_package = "my.package.name";
option java_multiple_files = true;

// ======== service definitions =================

service ServiceName {
  // Unary
  rpc Method1(RequestType) returns (ResponseType);

  // Server Streaming
  rpc Method2(RequestType) returns (stream ResponseType);

  // Client Streaming
  rpc Method3(stream RequestType) returns (ResponseType);

  // Bidirectional Streaming
  rpc Method4(stream RequestType) returns (stream ResponseType);
}

// ======= message definitions ==================

message RequestType {
  string name = 1;
  int32 id = 2;
  bool active = 3;
  int64 big_number = 4;
  double value = 5;
  bytes raw_data = 6;
}

// Liste
message ResponseType {
  repeated string values = 1;
}

// Map
message ExampleMap {
  map<string, int32> counts = 1;
}
```

```
// Verschachtelte Messages
message Outer {
  message Inner {
    string text = 1;
  }

  Inner inner = 1;
}

// ========== onof (union) ==============
message ExampleOneof {
  oneof payload {
    string text = 1;
    int32 number = 2;
    bytes data = 3;
  }
}

// ============ enum =============
enum ExampleEnum {
  UNKNOWN = 0;   // erster Wert muss 0 sein
  VALUE1 = 1;
  VALUE2 = 2;
}

// Enum in Message verwenden
message ExampleWithEnum {
  ExampleEnum state = 1;
}

// ========= empty message ================

import "google/protobuf/empty.proto";

// Beispiel:
rpc Method5(google.protobuf.Empty) returns (ResponseType);
```